МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Компьютерные науки и прикладная математика»

Лабораторная работа № 3
по курсу «Численные методы»

Группа: М8О-303Б-23

Студент(ка): Жуховицкий А. Д.

Преподаватель: Ревизников Д.Л.

Оценка:

Москва, 2026

ОГЛАВЛЕНИЕ

1 Тема.....3
2 Задание 3
3 Ход лабораторной работы.....4

1 Тема

Методы приближения функций. Численное дифференцирование и интегрирование.

2 Задание

3.1. Используя таблицу значений Y_i функции $y = f(x)$, вычисленных в точках X_i, $i = 0,...,3$ построить интерполяционные многочлены Лагранжа и Ньютона, проходящие через точки $\{X_i, Y_i\}$. Вычислить значение погрешности интерполяции в точке X^*.

7. $y = \sqrt{x}$, а) $X_i = 0, 1.7, 3.4, 5.1$; б) $X_i = 0, 1.7, 4.0, 5.1$; $X^* = 3.0$.

3.2. Построить кубический сплайн для функции, заданной в узлах интерполяции, предполагая, что сплайн имеет нулевую кривизну при $x = x_0$ и $x = x_4$. Вычислить значение функции в точке $x = X^*$.

7. $X^* = 3.0$

i	0	1	2	3	4
x_i	0.0	1.7	3.4	5.1	6.8
f_i	0.0	1.3038	1.8439	2.2583	2.6077

3.3. Для таблично заданной функции путем решения нормальной системы МНК найти приближающие многочлены а) 1-ой и б) 2-ой степени. Для каждого из приближающих многочленов вычислить сумму квадратов ошибок. Построить графики приближаемой функции и приближающих многочленов.

7.

i	0	1	2	3	4	5
x_i	0.0	0.2	0.4	0.6	0.8	1.0
y_i	1.0	1.0032	1.0512	1.2592	1.8192	3.0

3.4. Вычислить первую и вторую производную от таблично заданной функции $y_i = f(x_i)$, i = 0,1,2,3,4 в точке $x = X^*$.

7. $X^* = 0.2$

i	0	1	2	3	4
x_i	-0.2	0.0	0.2	0.4	0.6
y_i	1.7722	1.5708	1.3694	1.1593	0.9273

3.5. Вычислить определенный интеграл $F = \int_{X_0}^{X_k} y\,dx$, методами прямоугольников, трапеций, Симпсона с шагами h_1, h_2. Оценить погрешность вычислений, используя Метод Рунге-Ромберга:

7. $y = \dfrac{1}{3x^2 + 4x + 2}$, $X_0 = -2,\quad X_k = 2,\quad h_1 = 1.0,\quad h_2 = 0.5;$

3 Ход лабораторной работы

```
from math import prod

import numpy as np
import pandas as pd
from IPython.display import display

x_star = 3
y = lambda x: np.sqrt(x)
x_a = np.array([0, 1.7, 3.4, 5.1])
x_b = np.array([0, 1.7, 4, 5.1])
eps = 1e-3
n = len(x_a)
```

```
y_a = y(x_a)
y_b = y(x_b)
display(pd.DataFrame({"x": x_a, "y": y_a}), pd.DataFrame({"x": x_b, "y": y_b}))
```

	x	y
0	0.0	0.000000
1	1.7	1.303840
2	3.4	1.843909
3	5.1	2.258318

	x	y
0	0.0	0.000000
1	1.7	1.303840
2	4.0	2.000000
3	5.1	2.258318

Интерполяция Лагранжа

```
import sympy as sp

x = sp.Symbol('x')

L_a = sum(y_a[i] * prod((x - x_a[j]) / (x_a[i] - x_a[j]) for j in range(n) if j != i)
L_b = sum(y_b[i] * prod((x - x_b[j]) / (x_b[i] - x_b[j]) for j in range(n) if j != i)
```

```
derivative = sp.diff(sp.sqrt(x), x, n)

derivative
```

$$-\frac{15}{16x^{\frac{7}{2}}}$$

Максимум достигается в точке $x = 0$, так как функция убывает, а её производные положительны. Посчитать M не получится...

```
# Ma = max(float(derivative.subs(x, x_a[i]).evalf()) for i in range(n))
# Mb = max(float(derivative.subs(x, x_b[i]).evalf()) for i in range(n))
#
# w_a = prod(x - i for i in x_a)
# w_b = prod(x - i for i in x_b)
#
# error_a = Ma * abs(w_a) / factorial(n)
# error_b = Mb * abs(w_b) / factorial(n)
#
# error_a = float(error_a.subs(x, x_star).evalf())
# error_b = float(error_b.subs(x, x_star).evalf())
#
dif_a_lagrange = abs(y(x_star) - float(L_a.subs(x, x_star).evalf()))
dif_b_lagrange = abs(y(x_star) - float(L_b.subs(x, x_star).evalf()))
#
# dif_a <= error_a, dif_b <= error_b
```

```
dif_a_lagrange, dif_b_lagrange
```

```
(np.float64(0.01972677181660165), np.float64(0.04266258595181016))
```

Интерполяция Ньютона

```
from functools import lru_cache

@lru_cache(maxsize=None)
def divided_diff(f, *x_args):
    if len(x_args) == 1:
        return f(x_args[0])
    else:
        return (divided_diff(f, *x_args[1:]) - divided_diff(f, *x_args[:-1])) / (x_ar
```

```
def get_w(n, x_data):
    return prod(x - i for i in x_data[:n])

P_newton_a = sum(get_w(i, x_a) * divided_diff(y, *x_a[:i + 1]) for i in range(n))
P_newton_b = sum(get_w(i, x_b) * divided_diff(y, *x_b[:i + 1]) for i in range(n))
```

```
dif_a_newton = abs(y(x_star) - float(P_newton_a.subs(x, x_star).evalf()))
dif_b_newton = abs(y(x_star) - float(P_newton_b.subs(x, x_star).evalf()))
#
# dif_a <= error_a, dif_b <= error_b
```

```
dif_a_newton, dif_b_newton
```

```
(np.float64(0.019726771816601874), np.float64(0.04266258595180972))
```

Валидация

```
pts_a = [(sp.nsimplify(xi), sp.sqrt(sp.nsimplify(xi))) for xi in x_a]
a_validate = sp.expand(sp.interpolate(pts_a, x))

display(
    sp.expand(L_a),
    sp.N(sp.expand(a_validate), 15)
)
np.isclose(
    float(L_a.subs(x, x_star).evalf()),
    float(a_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.0216470834816846x^3 - 0.24254062240408x^2 + 1.11672397567224x$

$0.0216470834816847x^3 - 0.24254062240408x^2 + 1.11672397567224x$

```
np.True_
```

```
pts_b = [(sp.nsimplify(xi), sp.sqrt(sp.nsimplify(xi))) for xi in x_b]
b_validate = sp.expand(sp.interpolate(pts_b, x))

display(
    sp.expand(L_b),
    sp.N(sp.expand(b_validate), 16)
)
np.isclose(
    float(L_b.subs(x, x_star).evalf()),
    float(b_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.0188466177753101x^3 - 0.223497455600733x^2 + 1.09244393799797x$

$0.01884661777531005x^3 - 0.2234974556007327x^2 + 1.09244393799797x$

```
np.True_
```

```
display(
    sp.expand(P_newton_a),
    sp.N(sp.expand(a_validate), 15)
)
np.isclose(
    float(P_newton_a.subs(x, x_star).evalf()),
    float(a_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.0216470834816846x^3 - 0.24254062240408x^2 + 1.11672397567224x$

$0.0216470834816847x^3 - 0.24254062240408x^2 + 1.11672397567224x$

```
np.True_
```

```
display(
    sp.expand(P_newton_b),
    sp.N(sp.expand(b_validate), 15)
)
np.isclose(
    float(P_newton_b.subs(x, x_star).evalf()),
    float(b_validate.subs(x, x_star).evalf()),
    eps
)
```

$0.01884661777531x^3 - 0.223497455600733x^2 + 1.09244393799797x$

$0.01884661777531x^3 - 0.223497455600733x^2 + 1.09244393799797x$

```
np.True_
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Math

x_star = 3

eps = 1e-3
n = 5

x = np.array([0, 1.7, 3.4, 5.1, 6.8])
y = np.array([0, 1.3038, 1.8439, 2.2583, 2.6077])
```

```
h = np.concatenate(([0], np.diff(x)))

slau = np.zeros((n - 2, 3), dtype=float)
free = np.zeros(n - 2, dtype=float)
```

Интерполяция кубическими сплайнами

Составление СЛАУ для нахождения коэффициентов c_i в кубическом сплайне:

```
for i in range(1, n - 1):
    slau[i - 1, 0] = h[i]
    slau[i - 1, 1] = 2 * (h[i] + h[i + 1])
    slau[i - 1, 2] = h[i]
    free[i - 1] = 3 * (
        (y[i + 1] - y[i]) / h[i + 1] -
        (y[i] - y[i - 1]) / h[i]
    )
slau[0, 0] = slau[-1, 2] = 0
```

```
slau, free
```

```
(array([[0. , 6.8, 1.7],
        [1.7, 6.8, 1.7],
        [1.7, 6.8, 0. ]]),
 array([-1.34770588, -0.22182353, -0.11470588]))
```

Решим системы через метод прогонки для нахождения коэффициентов c_i. Используем реализацию из ЛР 1.2

```
P, Q = np.zeros(n - 1, dtype=float), np.zeros(n - 1, dtype=float)

P[0], Q[0] = 0, 0

for i in range(0, n - 2):
    P[i + 1] = -slau[i, 2] / (slau[i, 1] + slau[i, 0] * P[i])
    Q[i + 1] = (free[i] - slau[i, 0] * Q[i]) / (slau[i, 1] + slau[i, 0] * P[i])
```

```
c = np.zeros(n - 2, dtype=float)

for i in range(n - 2, 0, -1):
    if P[i] == 0:
        c[i - 1] = Q[i]
        continue
    c[i - 1] = P[i] * c[i] + Q[i]

c = np.concatenate(([0], c))
```

Найдём коэффициенты a_i, b_i, d_i для каждого отрезка сплайна.

```
a = y[:-1]


b = np.zeros(n - 1, dtype=float)

for i in range(1, n - 1):
    b[i - 1] = (y[i] - y[i - 1]) / h[i] - h[i] * (2 * c[i - 1] + c[i]) / 3

b[-1] = (y[-1] - y[-2]) / h[-1] - 2 * h[-1] * c[-1] / 3


d = np.zeros(n - 1, dtype=float)

for i in range(n - 2):
    d[i] = (c[i + 1] - c[i]) / (3 * h[i + 1])

d[-1] = -c[-1] / (3 * h[-1])
```

Выражение для сплайна на каждом отрезке $[x_i, x_{i+1}]$:

```
import sympy as sp

x_sym = sp.Symbol('x')
S = []
for i in range(n - 1):
    S.append(
        a[i]
        + b[i] * (x_sym - x[i])
        + c[i] * (x_sym - x[i]) ** 2
        + d[i] * (x_sym - x[i]) ** 3
    )
    display(Math(rf"S(x_{{{i}}}) = {sp.latex(sp.expand(S[i]))}"))
```

$S(x_0) = -0.0400457241720218x^3 + 0.882673319327731x$

$S(x_1) = 0.044783882411096x^3 - 0.4326309935739x^2 + 1.61814600840336x - 0.416767857142857$

$S(x_2) = -0.00923024919310289x^3 + 0.118313148788928x^2 - 0.255064075630254x + 1.70620357142857$

$S(x_3) = 0.0044920909540287x^3 - 0.0916386554621855x^2 + 0.815690126050424x - 0.114078571428579$

Значение сплайна в точке $x^* = 3$:

```
from bisect import bisect_left
i = max(0, bisect_left(x, x_star) - 1)

display(Math(rf"f({x_star}) = {float(S[i].subs(x_sym, x_star).evalf())}"))
```

$f(3) = 1.7531560510017157$

Валидация

Сравним графики функции и сплайна:

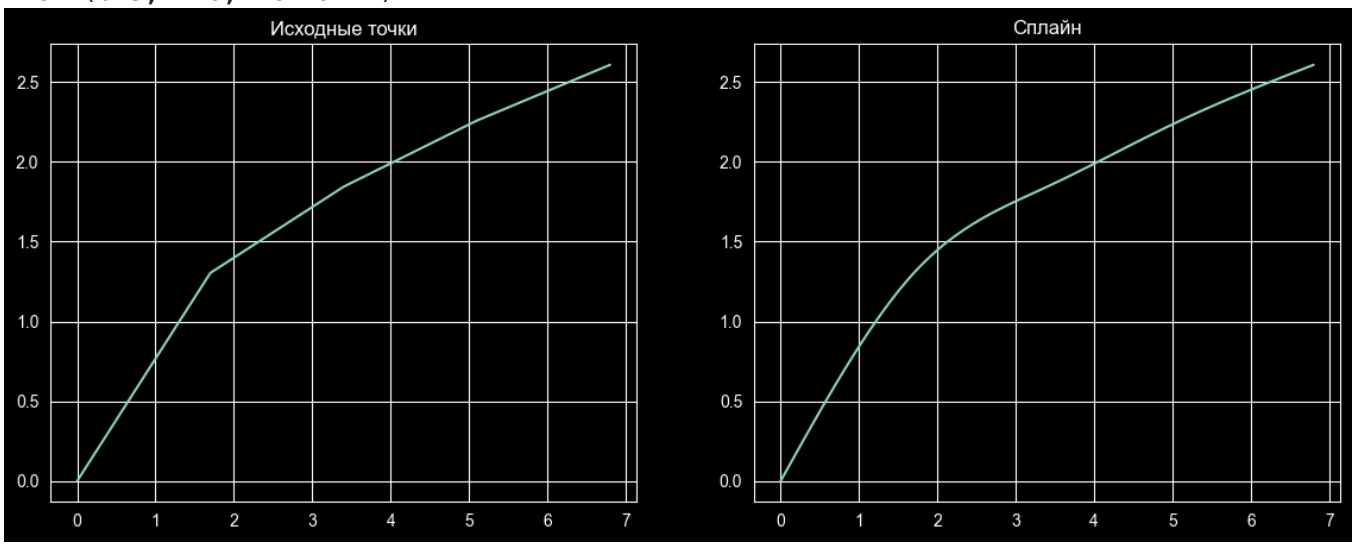
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.lineplot(x=x, y=y, ax=axes[0])
axes[0].set_title("Исходные точки")

spline_x = np.linspace(x[0], x[-1], 100)
spline_y = np.zeros(len(spline_x))
for i in range(len(spline_x)):
    idx = max(0, bisect_left(x, spline_x[i]) - 1)
    spline_y[i] = float(S[idx].subs(x_sym, spline_x[i]).evalf())

sns.lineplot(x=spline_x, y=spline_y, ax=axes[1])
axes[1].set_title("Сплайн")
```

```
Text(0.5, 1.0, 'Сплайн')
```




```
spline_first_derivative = [sp.diff(func, x_sym) for func in S]
spline_second_derivative = [sp.diff(func, x_sym, 2) for func in S]

for x_i in x[1:-1]:
    i = max(0, bisect_left(x, x_i) - 1)

    S_i = float(S[i].subs(x_sym, x_i))
```

```
    S_i_next = float(S[i + 1].subs(x_sym, x_i))

    spline_first_derivative_i = float(
        spline_first_derivative[i].subs(x_sym, x_i)
    )
    spline_first_derivative_i_next = float(
        spline_first_derivative[i + 1].subs(x_sym, x_i)
    )

    spline_second_derivative_i = float(
        spline_second_derivative[i].subs(x_sym, x_i)
    )
    spline_second_derivative_i_next = float(
        spline_second_derivative[i + 1].subs(x_sym, x_i)
    )

    print(
        f"Разрыва в точке {x_i} нет: {
        np.allclose(
            S_i,
            S_i_next,
            eps)
        } {
        np.allclose(
          spline_first_derivative_i,
          spline_first_derivative_i_next,
          eps
        )
        } {
        np.allclose(
          spline_second_derivative_i,
          spline_second_derivative_i_next,
          eps
        )
        }"
    )
```

```
Разрыва в точке 1.7 нет: True True True
Разрыва в точке 3.4 нет: True True True
Разрыва в точке 5.1 нет: True True True
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Math
import sympy as sp

eps = 1e-3
n = 6

x_sym = sp.Symbol('x')
x = np.array([0, 0.2, 0.4, 0.6, 0.8, 1])
y = np.array([1, 1.0032, 1.0512, 1.2592, 1.8192, 3])
```

Метод наименьших квадратов

Определим функцию, которая строит матрицу коэффициентов системы для метода наименьших квадратов для многочлена степени $order$:

```
def build_mnk_system(order):
    s = np.array([np.sum(x ** p) for p in range(2 * order + 1)], dtype=float)

    A = np.fromfunction(
        lambda k, i: s[(k + i).astype(int)],
        (order + 1, order + 1)
    )

    b = np.array([np.sum(y * (x ** k)) for k in range(order + 1)], dtype=float)

    return A, b
```

Многочлен 1-ой степени

Составим систему для многочлена 1-ой степени:

```
order = 1

A, b = build_mnk_system(order)
display(pd.DataFrame(A), pd.DataFrame(b))
```

	0	1
0	6.0	3.0
1	3.0	2.2

	0
0	9.1328
1	5.8320

Решим СЛАУ

```
from mtx_utils import lu_slau_solver

mnk_coefficients = lu_slau_solver(A, b)
display(pd.DataFrame(mnk_coefficients, columns=["a_i"]))
```

	a_i
0	0.618133
1	1.808000

Получившийся полиномиальный многочлен первой степени:

```
polynomial_first = sum(mnk_coefficients[i] * x_sym ** i for i in range(order + 1))

display(Math(rf"F_{{{order}}}(x) = {sp.latex(sp.expand(polynomial_first))}"))
```

$F_1(x) = 1.808x + 0.618133333333334$

Посчитаем сумму квадратов ошибок для многочлена первой степени:

```
sum(
    (float(polynomial_first.subs(x_sym, x[i]).evalf()) - y[i]) *
    (float(polynomial_first.subs(x_sym, x[i]).evalf()) - y[i])
    for i in range(len(x))
)
```

```
np.float64(0.8169608533333335)
```

Многочлен 2-ой степени

Составим систему для многочлена 2-ой степени:

```
order = 2

A, b = build_mnk_system(order)
display(pd.DataFrame(A), pd.DataFrame(b))
```

	0	1	2
0	6.0	3.0	2.2000
1	3.0	2.2	1.8000
2	2.2	1.8	1.5664

	0
0	9.13280
1	5.83200
2	4.82592

Решим СЛАУ

```
mnk_coefficients = lu_slau_solver(A, b)
display(pd.DataFrame(mnk_coefficients, columns=["a_i"]))
```

	a_i
0	1.090514
1	-1.734857
2	3.542857

Получившийся полиномиальный многочлен второй степени:

```
polynomial_second = sum(mnk_coefficients[i] * x_sym ** i for i in range(order + 1))

display(Math(rf"F_{{{order}}}(x) = {sp.latex(sp.expand(polynomial_second))}"))
```

$F_2(x) = 3.54285714285715x^2 - 1.73485714285714x + 1.09051428571429$

Посчитаем сумму квадратов ошибок для многочлена второй степени:

```
sum(
    (float(polynomial_second.subs(x_sym, x[i]).evalf()) - y[i]) *
    (float(polynomial_second.subs(x_sym, x[i]).evalf()) - y[i])
    for i in range(len(x))
)
```

```
np.float64(0.06719780571428584)
```

Визуализация

```
x_plot = np.linspace(x.min(), x.max(), 200)

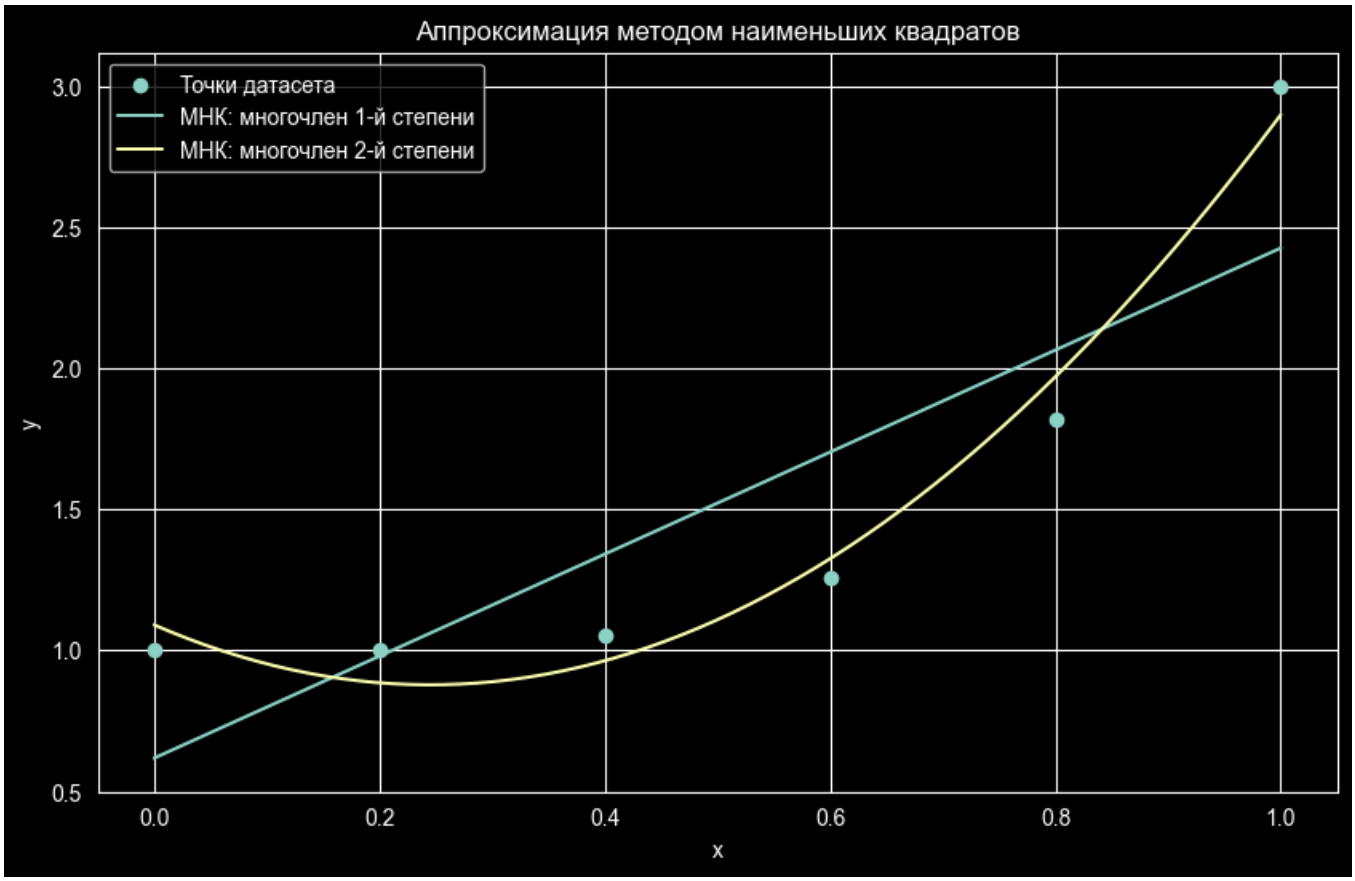
polynomial_first_func = sp.lambdify(x_sym, polynomial_first, modules="numpy")
```

```
polynomial_second_func = sp.lambdify(x_sym, polynomial_second, modules="numpy")

plt.figure(figsize=(10, 6))

plt.scatter(x, y, label="Точки датасета", zorder=3)
plt.plot(x_plot, polynomial_first_func(x_plot), label="МНК: многочлен 1-й степени")
plt.plot(x_plot, polynomial_second_func(x_plot), label="МНК: многочлен 2-й степени")

plt.title("Аппроксимация методом наименьших квадратов")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.legend()
plt.show()
```



Валидация

Используем готовые средства numpy для проверки получившихся многочленов. Сравним их с результатами, полученными вручную.

```
order = 1
mnk_numpy_coeffs_first = np.polynomial.polynomial.polyfit(x, y, order)
polynomial_numpy_first = sum(
    mnk_numpy_coeffs_first[i] * x_sym ** i
    for i in range(order + 1)
)
```

```
display(Math(rf"F_{{{order}}}(x) = {sp.latex(sp.expand(polynomial_first))}"))
display(Math(rf"F_{{{order}}}(x) = {sp.latex(sp.expand(polynomial_numpy_first))}"))
```

$F_1(x) = 1.808x + 0.618133333333334$

$F_1(x) = 1.808x + 0.618133333333333$

```
order = 2
mnk_numpy_coeffs_second = np.polynomial.polynomial.polyfit(x, y, order)
polynomial_numpy_second = sum(
    mnk_numpy_coeffs_second[i] * x_sym ** i
    for i in range(order + 1)
)

display(Math(rf"F_{{{order}}}(x) = {sp.latex(sp.expand(polynomial_second))}"))
display(Math(rf"F_{{{order}}}(x) = {sp.latex(sp.expand(polynomial_numpy_second))}"))
```

$F_2(x) = 3.54285714285715x^2 - 1.73485714285714x + 1.09051428571429$

$F_2(x) = 3.54285714285714x^2 - 1.73485714285714x + 1.09051428571429$

```
import numpy as np
from bisect import bisect_left
from IPython.display import display
import sympy as sp
from math import prod


x_sym = sp.Symbol('x')
x_star = 0.2
x = np.array([-0.2, 0, 0.2, 0.4, 0.6])
y = np.array([1.7722, 1.5708, 1.3694, 1.1593, 0.9273])
f = dict(zip(x, y))
eps = 1e-3
n = len(x)
```

Численное дифференцирование

Введём функцию для удобного вычисления разделённых разностей, которая будет использоваться при построении интерполяционного многочлена Ньютона. Кэширование результатов позволит избежать повторных вычислений для одних и тех же аргументов.

```
from functools import lru_cache


@lru_cache(maxsize=None)
def divided_diff(*x_args):
    if len(x_args) == 1:
        return f[x_args[0]]
    else:
        return (
            divided_diff(*x_args[1:])
            - divided_diff(*x_args[:-1])
        ) / (x_args[-1] - x_args[0])
```

Определим функции получения интерполяционного многочлена и произвдений $(x - x_i)$

```
def get_w(i, order, x_data):
    return prod(x_sym - xi for xi in x_data[i:i + order])


def get_interpolation_polynomial(i, order, x_data):
    phi = sum(
        get_w(i, k, x_data) * divided_diff(*x_data[i:i + k + 1])
        for k in range(order + 1)
    )
    return phi
```

Построим интерполяционные многочлены первой и второй степени для узлов, ближайших к x^*. Выберем индекс i так, чтобы $x[i] < x^* \leq x[i+1]$.

```
i = max(0, bisect_left(x, x_star) - 1)

phi1 = get_interpolation_polynomial(i, 1, x)
phi2 = get_interpolation_polynomial(i, 2, x)

display(sp.expand(phi1))
display(sp.expand(phi2))
```

$1.5708 - 1.007x$

$-0.108749999999999x^2 - 0.98525x + 1.5708$

Посмотрим на левостороннюю и правостороннюю производные в точке x^*, используя формулы для численного дифференцирования.

```
left_derivative = (y[i + 1] - y[i]) / (x[i + 1] - x[i])
right_derivative = (y[i + 2] - y[i + 1]) / (x[i + 2] - x[i + 1])

left_derivative, right_derivative
```

```
(np.float64(-1.0070000000000001), np.float64(-1.0504999999999998))
```

Теперь получим производную в точке x^*, используя интерполяционный многочлен первой степени.

```
phi1_derivative = float(sp.diff(phi1, x_sym).subs(x_sym, x_star).evalf())
phi1_derivative
```

```
-1.0070000000000001
```

Посчитаем вторую производную в точке x^*, используя формулу для численного дифференцирования второй производной

```
second_derivative = 2 * (
    (
        (
            (y[i + 2] - y[i + 1]) / (x[i + 2] - x[i + 1])
        ) - (
            (y[i + 1] - y[i]) / (x[i + 1] - x[i])
        )
    ) / (
        x[i + 2] - x[i]
    )
)
second_derivative
```

```
np.float64(-0.21749999999999825)
```

Посчитаем вторую производную в точке x^*, используя интерполяционный многочлен второй степени.

```
phi2_second_derivative = float(sp.diff(phi2, x_sym, 2).subs(x_sym, x_star).evalf())
```

```
phi2_second_derivative
```

```
-0.21749999999999825
```

Валидация

Сравним результаты численного дифференцирования и интерполяционного многочлена первой степени для первой производной в точке x^*.

```
np.isclose(phi1_derivative, left_derivative, eps)
```

```
np.True_
```

Сравним результаты численного дифференцирования и интерполяционного многочлена второй степени для второй производной в точке x^*.

```
np.isclose(phi2_second_derivative, second_derivative, eps)
```

```
np.True_
```

```
import numpy as np
from IPython.display import display, Math
import sympy as sp


x_0 = -2
x_k = 2
h_1 = 1.0
h_2 = 0.5
k = max(h_1, h_2) / min(h_1, h_2)
y = lambda x: 1 / (3 * x * x + 4 * x + 2)
x_var = sp.Symbol('x')
f = y(x_var)
eps = 5e-2
```

Метод прямоугольников

Определим функцию, которая считает приближённое значение интеграла по формуле метода прямоугольников для функции y на отрезке $[first_x, last_x]$ с фиксированным шагом h_step:

```
def get_rectangle(first_x, last_x, y, h_step):
    x_range = np.arange(first_x, last_x + h_step, h_step)

    return sum(
        h_step * y((x_range[i] + x_range[i + 1]) / 2)
        for i in range(len(x_range) - 1)
    )
```

Посчитаем приближённые значения интеграла по формуле метода прямоугольников для функции y на отрезке $[x_0, x_k]$ с шагами h_1 и h_2:

```
rectangle_h_1 = get_rectangle(x_0, x_k, y, h_1)
rectangle_h_1
```

```
np.float64(1.975292622928662)
```

```
rectangle_h_2 = get_rectangle(x_0, x_k, y, h_2)
rectangle_h_2
```

```
np.float64(1.8659258980586333)
```

Определим функцию, которая считает M - максимум по модулю от производной порядка $order$ для функции f на отрезке $[a, b]$:

```
def get_m(func, var, a, b, order):
    second_derivative = sp.diff(func, var, order)
    return max(
        abs(float(sp.maximum(second_derivative, var, sp.Interval(a, b)))),
        abs(float(sp.minimum(second_derivative, var, sp.Interval(a, b))))
    )
```

Найдём M_2 и M_4 для функции f на отрезке $[x_0, x_k]$:

```
m2 = get_m(f, x_var, x_0, x_k, 2)
m4 = get_m(f, x_var, x_0, x_k, 4)
```

Найдём верхнюю границу погрешности для метода прямоугольников для шагов h_1 и h_2:

```
rectangle_R_limit_h_1 = h_1 * h_1 * (x_k - x_0) * m2 / 24
rectangle_R_limit_h_2 = h_2 * h_2 * (x_k - x_0) * m2 / 24
```

Метод трапеций

Определим функцию, которая считает приближённое значение интеграла по формуле метода трапеций для функции y на отрезке $[first_x, last_x]$ с фиксированным шагом h_step:

```
def get_trapezoid(first_x, last_x, y, h_step):
    x_range = np.arange(first_x, last_x + h_step, h_step)

    return 0.5 * sum(
        h_step * (y(x_range[i]) + y(x_range[i + 1]))
        for i in range(len(x_range) - 1)
    )
```

Посчитаем приближённые значения интеграла по формуле метода трапеций для функции y на отрезке $[x_0, x_k]$ с шагами h_1 и h_2:

```
trapezoid_h_1 = get_trapezoid(x_0, x_k, y, h_1)
trapezoid_h_1
```

```
np.float64(1.7171717171717173)
```

```
trapezoid_h_2 = get_trapezoid(x_0, x_k, y, h_2)
trapezoid_h_2
```

```
np.float64(1.8462321700501896)
```

Найдём верхнюю границу погрешности для метода трапеций для шагов h_1 и h_2:

```
trapezoid_R_limit_h_1 = h_1 * h_1 * (x_k - x_0) * m2 / 12
trapezoid_R_limit_h_2 = h_2 * h_2 * (x_k - x_0) * m2 / 12
```

Метод Симпсона

Определим функцию, которая считает приближённое значение интеграла по формуле метода Симпсона для функции y на отрезке $[first_x, last_x]$ с фиксированным шагом h_step:

```
def get_simpson(first_x, last_x, y, h_step):
    x_range = np.arange(first_x, last_x + h_step, h_step)

    return (
        y(first_x) +
        y(last_x) +
        2 * sum(
            y(x_range[i])
            for i in range(1, len(x_range) - 2)
        ) +
        4 * sum(
            y((x_range[i - 1] + x_range[i]) * 0.5)
            for i in range(1, len(x_range) - 1)
        )
    ) * h_step / 6
```

Посчитаем приближённые значения интеграла по формуле метода Симпсона для функции y на отрезке $[x_0, x_k]$ с шагами h_1 и h_2:

```
simpson_h_1 = get_simpson(x_0, x_k, y, h_1)
simpson_h_1
```

```
np.float64(1.807017543859649)
```

```
simpson_h_2 = get_simpson(x_0, x_k, y, h_2)
simpson_h_2
```

```
np.float64(1.8297342810710984)
```

Найдём верхнюю границу погрешности для метода Симпсона для шагов h_1 и h_2:

```
simpson_R_limit_h_1 = h_1 * h_1 * h_1 * h_1 * (x_k - x_0) * m4 / 180
simpson_R_limit_h_2 = h_2 * h_2 * h_2 * h_2 * (x_k - x_0) * m4 / 180
```

Метод Рунге-Ромберга-Ричардсона

Определим функцию, которая считает приближённое значение погрешности по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений f_h и f_{kh} с порядком точности p:

```
def get_runge(f_h, f_kh, p, k_coef):
    return (f_h - f_kh) / (k_coef ** p - 1)
```

Найдём приближённые значения погрешности по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
runge_rectangle = get_runge(rectangle_h_2, rectangle_h_1, 2, k)
runge_trapezoid = get_runge(trapezoid_h_2, trapezoid_h_1, 2, k)
runge_simpson = get_runge(simpson_h_2, simpson_h_1, 4, k)
```

```
display(runge_rectangle, runge_trapezoid, runge_simpson)
```

```
np.float64(-0.03645557495667625)
np.float64(0.043020150959490744)
np.float64(0.0015144491474299601)
```

Найдём приближённые значения интеграла, улучшенные по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
rectangle_integral = rectangle_h_2 + runge_rectangle
print("Метод прямоугольников:")
display(Math(rf"F = \int_a^b f(x)\,dx = {rectangle_integral}"))
```

```
Метод прямоугольников:
```

$$F = \int_a^b f(x)\,dx = 1.8294703231019571$$

```
trapezoid_integral = trapezoid_h_2 + runge_trapezoid
print("Метод трапеция:")
display(Math(rf"F = \int_a^b f(x)\,dx = {trapezoid_integral}"))
```

```
Метод трапеция:
```

$$F = \int_a^b f(x)\,dx = 1.8892523210096803$$

```
simpson_integral = simpson_h_2 + runge_simpson
print("Метод прямоугольников:")
display(Math(rf"F = \int_a^b f(x)\,dx = {simpson_integral}"))
```

```
Метод прямоугольников:
```

$$F = \int_a^b f(x)\,dx = 1.8312487302185283$$

Валидация

Проверим, что приближённые значения погрешности, полученные по формуле метода Рунге-Ромберга-Ричардсона для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2, не меньше верхних границ погрешности для этих методов для шагов h_1 и h_2:

```
rectangle_R_limit_h_1 >= runge_rectangle and rectangle_R_limit_h_2 >= runge_rectangle
```

```
np.True_
```

```
trapezoid_R_limit_h_1 >= runge_trapezoid and trapezoid_R_limit_h_2 >= runge_trapezoid
```

```
np.True_
```

```
simpson_R_limit_h_1 >= runge_simpson and simpson_R_limit_h_2 >= runge_simpson
```

```
np.True_
```

Посчитаем точное значение интеграла функции f на отрезке $[x_0, x_k]$ и сравним его с приближёнными значениями, полученными по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
exact = float(sp.integrate(f, (x_var, x_0, x_k)))

display(Math(rf"F = \int_a^b f(x)\,dx = {exact}"))
```

$$F = \int_a^b f(x)\,dx = 1.8574186872187475$$

```
display(
    np.isclose(rectangle_integral, exact, eps),
    np.isclose(trapezoid_integral, exact, eps),
    np.isclose(simpson_integral, exact, eps)
)
```

```
np.True_
np.True_
np.True_
```

Посчитаем абсолютные погрешности для приближённых значений, полученных по формулам методов прямоугольников, трапеций и Симпсона для шагов h_1 и h_2:

```
print(
    f"Абсолютные погрешности:\n"
    f"Метод прямоугольников: {abs(rectangle_integral - exact)};\n"
    f"Метод трапеций: {abs(trapezoid_integral - exact)};\n"
    f"Метод Симпсона: {abs(simpson_integral - exact)};\n"
)
```

```
Абсолютные погрешности:
Метод прямоугольников: 0.02794836411679036;
Метод трапеций: 0.03183363379093285;
Метод Симпсона: 0.02616995700021918;
```